NEW Big News! We've announced ProcTHOR →

Material Randomization
Lighting Randomization
Initial Random Spawn
Color Randomization

Material Randomization

Randomizes all the materials in the scene. Each object may only randomize to sensible materials from a similar material type. For instance, an `Apple`'s materials won't change to an `Orange`'s materials, and a `Chair`'s material won't change to a `Plant`'s material. Empirically, the rooms generated from material randomization look incredibly realistic in most cases. Upon calling reset, materials change back to their original values.

```
controller.step(
    action="RandomizeMaterials",
    useTrainMaterials=None,
    useValMaterials=None,
    useTestMaterials=None,
    inRoomTypes=None
)
```

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Test it live on the demo →

Randomize Materials Parameters

useTrainMaterials: Optional[bool] = None

Specifies if the randomization should include materials found in training rooms. Training rooms consist of all of the following:

- Kitchens: $\text{FloorPlan}[1:20]$
- Living Rooms: $\text{FloorPlan}[201:220]$
- Bedrooms: $\text{FloorPlan}[301:320]$
- Bathrooms: $\text{FloorPlan}[401:420]$
- RoboTHOR Train: $\text{FloorPlan_Train}*_*$

If the agent is in one of the above scenes and `None` is passed in, then `useTrainMaterials` will default to `True`. Otherwise, it will default to `False`.

An exception is thrown if you are in a train scene, and you pass in `useTrainMaterials` as `False`. We do this because it guarantees every object can randomize to at least one material.

useValMaterials: Optional[bool] = None

Specifies if the randomization should include materials found in validation rooms. Validation rooms consist of all of the following:

- Kitchens: $\text{FloorPlan}[21:25]$
- Living Rooms: $\text{FloorPlan}[221:225]$
- Bedrooms: $\text{FloorPlan}[321:325]$
- Bathrooms: $\text{FloorPlan}[421:425]$
- RoboTHOR Val: $\text{FloorPlan_Val}*_*$

If the agent is in one of the above scenes and `None` is passed in, then `useValMaterials` will default to `True`. Otherwise, it will default to `False`.

An exception is thrown if you are in a validation scene, and you pass in `useValMaterials` as `False`. We do this because it guarantees every object can randomize to at least one material.

useTestMaterials: Optional[bool] = None

Specifies if the randomization should include materials found in testing rooms. Testing rooms consist of all of the following:

- Kitchens: $\text{FloorPlan}[26:30]$
- Living Rooms: $\text{FloorPlan}[226:230]$
- Bedrooms: $\text{FloorPlan}[326:330]$
- Bathrooms: $\text{FloorPlan}[426:430]$

If the agent is in one of the above scenes and `None` is passed in, then `useTestMaterials` will default to `True`. Otherwise, it will default to `False`.

An exception is thrown if you are in a testing scene, and you pass in `useTestMaterials` as `False`. We do this because it guarantees every object can randomize to at least one material.

inRoomTypes: List[str] = None

Specifies which room types the randomizable materials may come from.

Valid room types include `Kitchen`, `LivingRoom`, `Bedroom`, `Bathroom`, and `RoboTHOR`. Casing is ignored. An exception is thrown if an unknown room type is passed in.

If `None` is passed in, then all room types will be included.

An exception is thrown if the current room type is not included in `inRoomTypes` and `None` is not passed in. We do this because it guarantees every object can randomize to at least one material.

Lighting Randomization

```
controller.step(
    action="RandomizeLighting",
    brightness=(0.5, 1.5),
    randomizeColor=True,
    hue=(0, 1),
    saturation=(0.5, 1),
    synchronized=False
)
```

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Test it live on the demo →

Randomize Lighting Parameters

brightness: Tuple[float, float] = (0.5, 1.5)

Sets the bounds with which a light's intensity may be multiplied by. Higher values are brighter. Both values must be greater than 0. The order of the values does not matter.

randomizeColor: bool = True

Specifies if the color should be randomized. If set to `False`, only each lights intensity may change.

hue: Tuple[float, float] = (0, 1)

Sets the $(\min, \max)$ domain of possible hue values for each light, which affects its color. Valid values are ranges between $[0:1]$, where:

- 0 maps to a hue of 0 degrees (i.e., red).
- 0.5 maps to a hue of 180 degrees (i.e., green).
- 1 maps to a hue of 360 degrees (i.e., red).

saturation: Tuple[float, float] = (0.5, 1)

Sets the $(\min, \max)$ domain of possible saturation values for each light. Valid values are ranges between $[0:1]$, where 0 corresponds to grayscale and 1 corresponds to full saturation.

synchronized: bool = False

Specifies if all lights in the room should be randomized with the same values (i.e., intensity and color). When `False`, each lighting object gets its own independent randomized intensity and randomized color.

Initial Random Spawn

`InitialRandomSpawn` attempts to randomize the position of `Pickupable` objects, placing them in any valid receptacle they could be placed in within the scene. Objects are not guaranteed to move from their default position, as sometimes there is not enough free space in the scene for an object to move.

> **Warning**
> After calling initial random spawn, each object will have its objectId recomputed (in the form of "objectType|x|y|z"). Thus, objects that move will have their objectId's changed. This is necessary to prevent collisions between objectId's, since initial random spawn may be used to duplicate existing objects in the scene.

```
controller.step(
    action="InitialRandomSpawn",
    randomSeed=0,
    forceVisible=False,
    numPlacementAttempts=5,
    placeStationary=True,
    numDuplicatesOfType = [
        {
            "objectType": "Statue"
             "count": 20
        },
        {
            "objectType": "Bowl"
            "count": 20,
        }
    ],
    excludedReceptacles=["CounterTop", "DiningTable"],
    excludedObjectIds=["Apple|1|1|2"]
)
```

Initial Random Spawn Parameters

randomSeed: int = 0

Used to seed the randomization for duplicatable scene states. Because this seed depends on the current state of the scene, remember to reset the scene with controller.reset() before running InitialRandomSpawn(), otherwise the seeded randomization will not be accurate.

forceVisible: bool = False

When enabled, the scene will attempt to randomize all moveable objects outside of receptacles in plain view. Use this if you want to avoid objects spawning out of view inside closed drawers, cabinets, etc.

numPlacementAttempts: int = 5

How many times each object in the scene attempts to randomly spawn. Setting this value higher will lead to fewer spawn failures at the cost of performance.

placeStationary: bool = True

Determines if spawned objects will be settled completely static and unmoving, or if non-determenistic physics resolve their final position. Setting this to False will allow physics to resolve final positions, which can be used to spawn an object on a sloped receptacle but have it end up rolling off.

numDuplicatesOfType: list[dict[str, any]] = []

Used to specify how many objects of a certain type will attempt to be duplicated somewhere in the scene. It does not guarantee this number of duplicated objects, only the number of attempted spawned objects, so this is the max it will be. This will only create copies of objects already in the scene, so if you request an object which is not in the scene upon reset, it will not work.

excludedReceptacles: list[str] = []

A list of receptacle object types to exclude from valid receptacles that can be randomly chosen as a spawn location. An example use case is excluding all CounterTop receptacles to allow for a scene configuration that has more free space on CounterTops in case you need free space for interaction. Note that this will not guarantee all listed receptacles as being completely clear of objects, as any objects that failed to reposition will remain in their default position, which might have been on the excluded receptacle type. Check the Actionable Properties section of the Objects documentation for a full list of Receptacle objects.

excludedObjectIds: list[str] = []

Used to prevent certain objects from changing in position.

> **Remark**
> This is often nice for rearrangement related tasks.

Color Randomization

```
controller.step(action="RandomizeColors")
```

Randomize Colors Description

Randomizes the color of each object in the scene. Compared to `RandomizeMaterials`, `RandomizeColors` looks much less realistic, but does enable a huge potential for visual variance.

Each color is uniformly sampled with a hue between $[0:1]$, a saturation in $[0.5:1]$, and a value in $[0.5:1]$.

이 동영상을 시청하려면 Vimeo에 로그인하세요.
먼저 쿠키를 허용해야 할 수도 있습니다. 메시지가 표시되면 허용을 선택하여 계속합니다.
로그인

Test it live on the demo →